

MAD 2 Project Report

1. Student Details

Name: Saboo Riddesh Rajan

Roll Number: DS24F1001102

Email: 24f1001102@ds.study.iitm.ac.in

About Me: I am a Diploma level student of the IIT Madras BS Degree program interested in full-stack web application development and Mobile application development. I enjoy building applications that can be used in real life applications.

2. Project Details

Project Title: Placement Portal Application - V2

Problem Statement:

Institutes require efficient systems to manage campus recruitment activities involving companies and students. Currently, many institutes rely on spreadsheets, emails, or manual coordination, which makes it difficult to manage company approvals, placement drives, student registrations, and application tracking. The objective is to build a **Placement Portal Application (PPA)** web application that allows **Admin (Institute), Companies, and Students** to interact with the system based on their roles.

Approach:

The app was built using Flask as the backend framework, the Flask backend was divided into four main Blueprints namely 'auth', 'admin', 'student' and 'company'. SQLAlchemy was used as ORM (Object Relational Mapper) with SQLite as the database. Redis and Flask-Caching were used for caching commonly accessed routes. Redis and Celery were used for execution of batch jobs such as CSV export generation, monthly placement reports, and interview reminder emails.

Vue.js was used for developing the frontend, Bootstrap was used for styling and responsiveness, Chart.js was used for generating charts for visualization of data. An ATS-style Resume Match Analysis module was also implemented to compare resumes against job requirements before and during the application review process.

3. AI/LLM Declaration

I used ChatGPT (GPT-5.5) for understanding concepts, resolving programming issues, documentation, preparing the API YAML file, and improving the README file.

The extent of AI/LLM usage is approximately 10–15%. All application design decisions, database schema design, feature implementation, debugging, testing, integration, and final code modifications were completed manually by me.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend web framework for building REST APIs
SQLAlchemy	Object Relational Mapper for SQLite database
SQLite	Relational database for storing application data

Vue.js	Frontend framework for building the Single Page Application
Vue Router	Client-side routing and navigation
Axios	Communication between Vue frontend and Flask REST APIs
Bootstrap 5	Frontend styling and responsive design
Chart.js	Data analytics and visualization
Flask-JWT-Extended	JWT-based authentication and role-based authorization
Flask-Caching	Caching frequently accessed API responses using Redis
Redis	In-memory data store used for caching and Celery backend
Celery	Background task execution and scheduled jobs
Flask-Mail	Sending interview reminders and monthly placement reports via email
pdfplumber	Parsing PDF resumes for Resume Match Analysis (ATS)
Werkzeug	Password hashing and security utilities

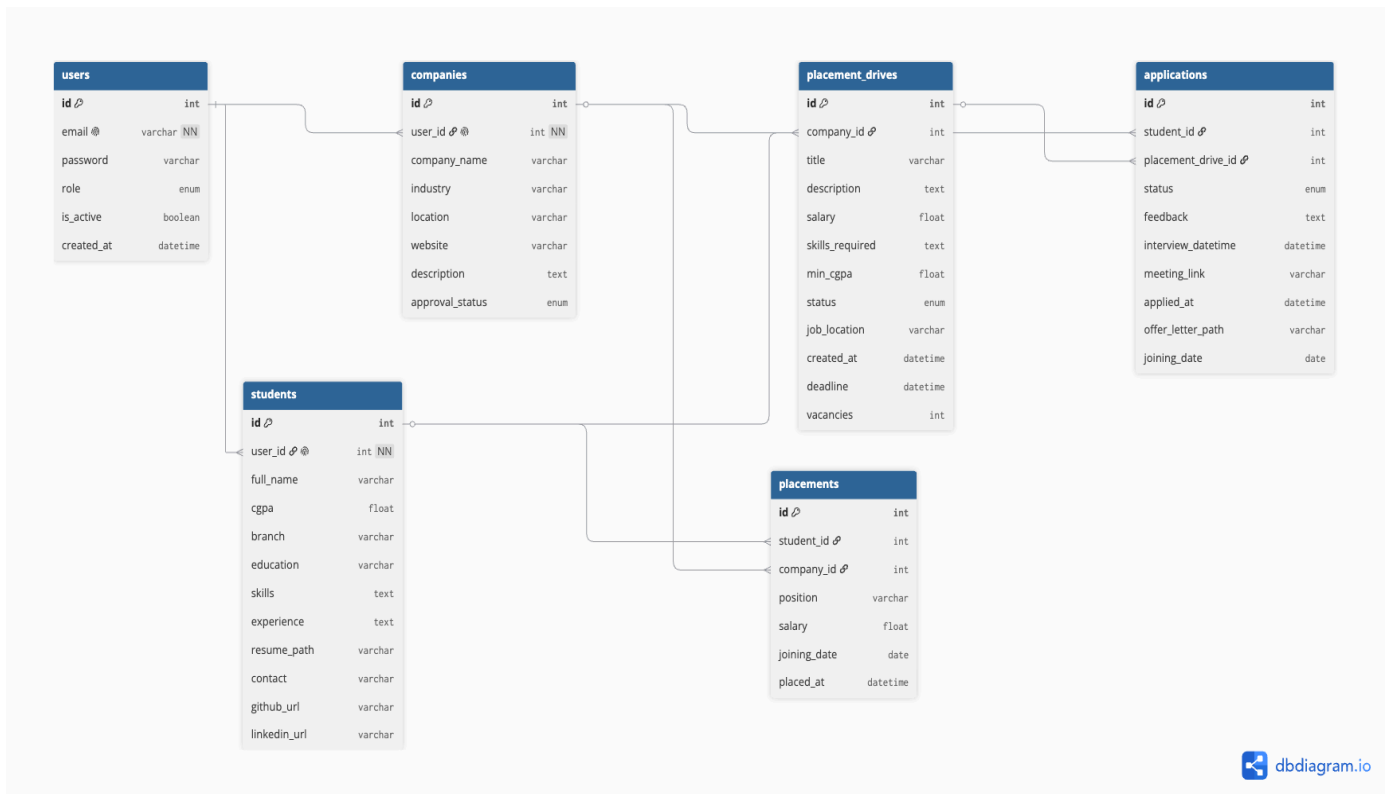
5. Database Schema / ER Diagram

Tables

1. **User** – Stores user authentication details including email, password, role, account status, and account creation date.
2. **Student** – Stores student profile details such as name, branch, CGPA, education, skills, experience, resume, contact information, and social profile links.
3. **Company** – Stores company information including company name, industry, location, website, description, and approval status.
4. **PlacementDrive** – Stores placement drive information such as job title, description, package, required skills, eligibility criteria, vacancies, deadline, and drive status.
5. **Application** – Stores applications submitted by students along with application status, interview details, feedback, offer letter, and joining date.
6. **Placement** – Stores final placement records of students including company, position, salary, and joining date.

Relationships

- **One-to-One** → User → Student
- **One-to-One** → User → Company
- **One-to-Many** → Company → PlacementDrive
- **One-to-Many** → Student → Application
- **One-to-Many** → PlacementDrive → Application
- **One-to-Many** → Student → Placement
- **One-to-Many** → Company → Placement



6. API Resource Endpoints

The Placement Portal Application uses RESTful APIs to facilitate communication between Students, Companies, Administrators and the Public Landing Dashboard. APIs are organized into four Flask Blueprints (auth, student, company and admin) and secured using JWT authentication wherever needed.

1. Authentication APIs

Purpose: Handles user registration, authentication and public access.

- POST `/register/student` – Register a new student account.
- POST `/register/company` – Register a new company account.
- POST `/login` – Authenticate user and generate JWT token.
- GET `/public/dashboard` – Retrieve public dashboard statistics.

2. Student APIs

Purpose: Allows students to manage profiles, placement drives and applications.

- GET `/student/dashboard/{student_id}` – Retrieve student dashboard.
- GET `/student/profile` – View student profile.
- PUT `/student/profile/update` – Update student profile.
- GET `/student/job_postings` – List approved placement drives.
- GET `/student/job_posting/{posting_id}` – View placement drive details.
- POST `/student/job_posting/{posting_id}/apply` – Apply for placement drive.
- GET `/student/applications` – View submitted applications.
- GET `/student/application/{application_id}` – View application details.
- PUT `/student/application/{application_id}/accept` – Accept placement offer.

- PUT /student/application/{application_id}/reject – Reject placement offer.
- GET /student/resume – Download uploaded resume.
- GET /student/application/{application_id}/offer_letter – Download offer letter.
- POST /student/export – Generate CSV export report.
- GET /student/export/download – Download generated CSV report.
- GET /student/resume_screener/{placement_drive_id} – Perform Resume Match Analysis.

3. Company APIs

Purpose: Allows companies to manage placement drives and review candidates.

- POST /company/addjobposting – Create a new placement drive.
- GET /company/dashboard/{company_id} – Retrieve company dashboard.
- GET /company/job_postings/{company_id} – View company placement drives.
- GET /company/job_posting/{posting_id} – View placement drive details.
- PUT /company/job_posting/{posting_id} – Update placement drive details.
- PUT /company/job_posting/{posting_id}/status – Close placement drive.
- GET /company/job_posting/{posting_id}/applications – View received applications.
- GET /company/application/{application_id} – View application details.
- PUT /company/application/{application_id}/shortlist – Shortlist candidate.
- PUT /company/application/{application_id}/reject – Reject candidate.
- PUT /company/application/{application_id}/interview_scheduled – Schedule interview.
- PUT /company/application/{application_id}/select – Select candidate.
- GET /company/student/{student_id}/resume – Download student resume.
- GET /company/application/{application_id}/offer_letter – Download offer letter.
- POST /company/export – Generate CSV export report.
- GET /company/export/download – Download generated CSV report.
- POST /company/resume_screener/{application_id} – Perform Resume Match Analysis.

4. Administrator APIs

Purpose: Enables complete administration of the Placement Portal.

- GET /admin/dashboard – Retrieve administrator dashboard.
- GET /admin/companies – View all companies.
- PUT /admin/company/{company_id}/status – Approve, reject or blacklist company.
- GET /admin/students – View all students.
- PUT /admin/student/{student_id}/status – Activate or blacklist student.
- GET /admin/job_postings – View all placement drives.
- PUT /admin/job_posting/{posting_id}/status – Approve or reject placement drive.
- GET /admin/applications – View all submitted applications.
- GET /admin/student/{student_id} – View student profile.
- GET /admin/student/resume/{student_id} – Download student resume.
- GET /admin/company/{company_id} – View company profile.
- GET /admin/application/{application_id}/offer_letter – Download offer letter.
- GET /admin/student/applications/{student_id} – View student applications.
- GET /admin/company/job_postings/{company_id} – View company placement drives.
- POST /admin/reminders – Trigger interview reminder emails.
- POST /admin/monthly_report/{current_month} – Generate monthly placement report.

YAML API Definition File: [api.yaml](#)

Also included in the submission ZIP as `api.yaml`.

7. Architecture and Features

Architecture Overview:

The application follows a client-server architecture with Vue.js as the frontend and Flask as the backend. The backend is organized into four Flask Blueprints, namely auth, admin, student, and company, to provide modular role-based functionality. SQLAlchemy is used for database operations with SQLite as the relational database. Redis and Flask-Caching are used to cache frequently accessed data, while Celery with Redis executes scheduled background tasks such as CSV export generation, monthly placement reports, and interview reminder emails. The frontend communicates with the backend through RESTful APIs using Axios, while Bootstrap 5 provides a responsive user interface and Chart.js is used for data visualization. An ATS-style Resume Match Analysis module compares uploaded resumes against job requirements for students and companies.

- **app.py** – Flask application entry point and Blueprint registration.
- **models.py** – SQLAlchemy database models.
- **routes/** – Flask Blueprints for Authentication, Student, Company and Administrator APIs.
- **frontend/** – Vue.js components, views and routing.
- **tasks.py** – Celery background tasks for reports and interview reminders.
- **celery_app.py** – Celery configuration and scheduling.
- **uploads/** – Stores resumes and offer letters.
- **exports/** – Stores generated CSV reports.

Implemented Features:

- Student, Company and Administrator role-based authentication using JWT.
- Company approval and placement drive management.
- Student profile management with resume upload.
- Placement drive application and application tracking.
- Resume Match Analysis (ATS-style) for students and companies.
- Public landing page dashboard with Chart.js analytics.
- Dashboard statistics and placement analytics.
- CSV export of applications and reports.
- Automated interview reminder emails and monthly placement reports using Celery.
- Redis caching for improved API performance.
- Responsive user interface developed using Bootstrap 5.

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1b5gHaIDZTW2rKeGC3S45AQcLBNHpcu-U/view?usp=sharing>

 2026-07-13 14-16-31.mov